Data Parser script

```
% Author: Jiale Jiang
% Editor: Jiale Jiang(self-edited)
```

Program Purpose:

parse all raw data into more organized struct fields and cell arrays for better human readablity, then do some neccessary calculation/analysis/unit conversion to clean up the data then plot all required data plots for the final deliverable, all plots will be exported and saved into the designated folder;

Load all the saved variables in the workspace

```
clearvars;
load("JettaData.mat");
load("CamryData.mat");

% These boolean flags were initially used to manually select which plot to
% plot, later on these flags are automated by a for loop I made to plot all
% plots one by one without having to do it manually.

print_jetta_table = 0;
print_camry_table = 0;
show_ave_idle_rpms = 1;
plotting_enabled = 0;

plot_camry_accel = 0;
plot_camry_raw_accel = 0;
plot_camry_vel = 0;
plot_camry_filtered_vel = 0;
plot_camry_pos = 0;
plot_camry_rpmtime = 0;
plot_camry_rpmtime_idle1 = 0;
plot_camry_rpmtime_accel = 0;
plot_camry_rpmtime_decel = 0;
plot_camry_rpmtime_idle2 = 0;
plot_camry_rpmvel = 0;
plot_camry_rpmvel_idle1 = 0;
plot_camry_rpmvel_accel = 0;
plot_camry_rpmvel_decel = 0;
plot_camry_rpmvel_idle2 = 0;
plot_camry_fuelratetime = 0;
plot_camry_fuelused = 0;

plot_jetta_pos = 0;
plot_jetta_vel = 0;
plot_jetta_accel = 0;
plot_jetta_filtered_accel = 0;
plot_jetta_rpmtime = 0;
plot_jetta_rpmtime_idle1 = 0;
plot_jetta_rpmtime_accel = 0;
```

```
plot_jetta_rpmtime_decel = 0;
plot_jetta_rpmtime_idle2 = 0;
plot_jetta_rpmvel_idle1 = 0;
plot_jetta_rpmvel_accel = 0;
plot_jetta_rpmvel_decel = 0;
plot_jetta_rpmvel_idle2 = 0;
plot_jetta_fuelratetime = 0;
plot_jetta_fuelused = 0;
```

1. Data Parsing

a. load data from previously saved workspace the data was imported as numerical matrix, following data parsing can help our code to be way easier to work with and more human readable;

```
% b.
% create a cell containing 3 raw data matricies
RawJettaData = {Jetta1, Jetta2, Jetta3};
RawCamryData = {CamryData1 CamryData2 CamryData3};

% create a vector of string names for struct name parsing
newNameJetta = ["Time_sec", "TripDistance_mile", "VehicleSpeed_MPH", ...
    "Acceleration_fts2", "TripFuel_gal", "TotalFuelEcon_MPG", ...
    "InstantFuelEcon_MPG", "EngineRPM_rpm"];
newNameCamry = ["Time_sec","EngineRPM_rpm","VehicleSpeed_MPH",...
    "FuelRate_lb_per_min"];
% c.
% parse raw data into two cells;
JettaData = parsingData(newNameJetta,RawJettaData);
CamryData = parsingData(newNameCamry,RawCamryData);

% some unit conversion
JettaData = convertUnits(JettaData);
CamryData = convertUnits(CamryData);
for i = 1:3
    JettaData{i}.trip_distanceft = ...
        JettaData{i}.TripDistance_mile  .* 5280;
    JettaData{i}.instantFuelRategals = ...
        (JettaData{i}.VehicleSpeed_MPH ./ ...
        JettaData{i}.InstantFuelEcon_MPG)./3600;
end

% This is to create a vector of consistent color for each trial and help
% with legend writing
JettaTrialColors = ["b","r","c"];
CamryTrialColors = ["b","r","c"];
TrialLegends = ["Trial 1","Trial 2","Trial 3"];
```

2. Compute Data Sampling Frequency

```
% a. Frequency calculation
JettaTimeData = cell(1,3);
CamryTimeData = cell(1,3);
```

```
for i = 1:3
    JettaTimeData{i} = calculateTimeData(JettaData{i}.Time_sec);
    CamryTimeData{i} = calculateTimeData(CamryData{i}.Time_sec);
end
```

3. Data Presentation as table output in MAT-LAB terminal

```
% a. 2022 Volkswagen Jetta SE and 2024 Toyota Camry SE
% Test Procedure: 1)Park at a relatively flat parking lot;
%                 2)Accelerate the car from 0-35 mph;
%                 3)Once reach 35 mph,deccelerate the car to 0 mph safely;

% These table print out are only for the OBD2 data collection assignment,
% disabled for the project data analysis by the flag;
if print_jetta_table == 1
    printJettaDataTable(JettaData);
end
if print_camry_table == 1
    printCamryDataTable(CamryData);
end
```

Average sampling frequency for Jetta trials 1-3:[8.23,7.86,8.46] Hz
Average sampling frequency for Camry trials 1-3:[19.99,20.00,20.00] Hz

4. Helper Function Declaration

```
% this function takes in a string vector, and a cell
function parsedData = parsingData(newName,RawData)
    % the parsed data should be a cell too
    parsedData = cell(size(RawData));
    for i = 1:size(RawData,2)
       % the parsed data contains structs, same size as raw data cell
        parsedData{i} = struct();
        % for each element in the name field:
        for j = 1:numel(newName)
            % the chosen named struct object = corresponding raw data
            % column
            parsedData{i}.(newName(j)) = RawData{i}(:, j);
        end
    end
end

% this function calculate the average frequency of the data in each trial
function TimeData = calculateTimeData(timeVector)
    N = numel(timeVector);
    total_time = timeVector(end) - timeVector(1);
    TimeData.aveInterval = total_time/(N-1);
    TimeData.aveFrequency = 1/TimeData.aveInterval;
end

% this function prints 3 tables of data given a data cell containing 3
```

```
% trials
function table = printJettaDataTable(Datacell)
    for i = 1:size(Datacell,2)
        fprintf("\nJetta Trial %d\n",i);
        fprintf("%5s %4s %8s %3.5s %13s %7s %5s %5s\n", ...
    "Time(s)", "Speed(mph)", "Accel(ft/s^2)", ...
    "RPM", "TripDist(mi)", "InstMPG","TripFuel","TotalMPG");

        table = [Datacell{i}.Time_sec, Datacell{i}.VehicleSpeed_MPH, ...
                Datacell{i}.Acceleration_fts2, ...
                Datacell{i}.EngineRPM_rpm, ...
                Datacell{i}.TripDistance_mile, ...
                Datacell{i}.InstantFuelEcon_MPG,...
                Datacell{i}.TripFuel_gal,...
                Datacell{i}.TotalFuelEcon_MPG];
        fprintf("%5.3f %8.2f %12.3f %9.1f %10.5f %7.2f " + ...
            "%8.2f %8.2f\n", table.');
    end
end

% a similar table printing function to print data tables
function table = printCamryDataTable(Datacell)
    for i = 1:size(Datacell,2)
        fprintf("\nCamry Trial %d\n",i);
        fprintf("%5s %12s %12s %12s \n", ...
    "Time(s)","RPM", "Speed(MPH)", "Fuel Rate(lb per min)");

        table = [Datacell{i}.Time_sec,...
                Datacell{i}.EngineRPM_rpm, ...
                Datacell{i}.VehicleSpeed_MPH,...
                Datacell{i}.FuelRate_lb_per_min];
        fprintf("%5.3f %12.2f %12.4f %12.5f\n", table.');
    end
end

% prints the sampling frequency of the 3 trials
fprintf("Average sampling frequency for Jetta trials 1-3:" + ...
    "[%.2f,%.2f,%.2f] Hz\n",[JettaTimeData{1}.aveFrequency,...
    JettaTimeData{2}.aveFrequency,...
    JettaTimeData{3}.aveFrequency]);
 fprintf("Average sampling frequency for Camry trials 1-3:" + ...
    "[%.2f,%.2f,%.2f] Hz\n",[CamryTimeData{1}.aveFrequency,...
    CamryTimeData{2}.aveFrequency,...
    CamryTimeData{3}.aveFrequency]);

% unit conversion function
function Datacell_unit_converted = convertUnits(Datacell)
    for i = 1:length(Datacell)
        Datacell{i}.speed_fts = ...
            Datacell{i}.VehicleSpeed_MPH .* 5280 ./ 3600;
        if ~isfield(Datacell{i}, 'FuelRate_lb_per_min')
        continue;
        end
        Datacell{i}.fuelrate_lbs = ...
```

```
        Datacell{i}.FuelRate_lb_per_min ./60;
    end
        Datacell_unit_converted = Datacell;
end
```

Calculating acceleration with Kalman Filter

I made the Kalman Filter function in another script file and saved it as a function file only, more details about Kalman Filter will be showed in the final report and my comments in the actual function;

```
% motion detection function is also written by me to use some simple speed
% detection logic to identify the idle, acceleration and deceleration
% sections of the data, it does not work perfectly, but it work good enough
% to give us good data, you will see it in the graphs;

% acceleration calculation and motion detection

% Jetta/Camry motions are cell array to contain the returned indices of the
% motion section, for example, if the idle time of a car is 0-6 sec, then
% the cell array will use its internal struct field to contain the
% corresponding indices, and so we can use those indices to find the
% corresponding sections of our desired motion (accel or decel or idle)
% more comments about this motion detection function will be seen in our
% motion detection function file

JettaMotion = cell(1,3);
CamryMotion = cell(1,3);

% very classic for loop, just used so that we don't have to call it one by
% one,

% logic is simple, for each car's each trial, we parse the needed argument
% into the kalmand filter function, then we parse its returned speed and
% accel back into each of our cell array's struct field for more organized
% data storage;

% the cumtrapz() function is used to integrate the distance traveled from
% speed, it will use the mean velocity approximation as the instaneous
% velocity between two time stamp, then do discrete integration to find the
% accumulated distance traveled in each trial, we only used this on Camry
% since we didn't record its distance traveled data;

for i = 1:3
    % call Kalman Filter
    filtered_camry_data = ...
        SpeedKalmanFilter(CamryData{i}.Time_sec,...
        CamryData{i}.speed_fts);
    % save results in struct fields
    CamryData{i}.filtered_speed = filtered_camry_data.filtered_speed;
    CamryData{i}.filtered_accel = filtered_camry_data.predicted_accel;

    % call the cumtrapz to integrate the distance from speed
    CamryData{i}.trip_distanceft = cumtrapz(CamryData{i}.Time_sec,...
```

```
        CamryData{i}.speed_fts);
    % save results in struct fields
    CamryData{i}.filtered_trip_distnaceft = ...
        cumtrapz(CamryData{i}.Time_sec,...
        CamryData{i}.filtered_speed);
    % call the cumtrapz to integrate cumulated fuel usage and save in
    % struct field
    CamryData{i}.cumulated_fuel_lb = ...
        cumtrapz(CamryData{i}.Time_sec,CamryData{i}.fuelrate_lbs);

    % call Kalman Filter for Jetta
    filtered_jetta_data = SpeedKalmanFilter(JettaData{i}.Time_sec,...
        JettaData{i}.speed_fts);

    JettaData{i}.filtered_accel = ...
        filtered_jetta_data.predicted_accel;
    JettaData{i}.filtered_speed = ...
        filtered_jetta_data.filtered_speed;

    % call motion detection function, and save all objects that contains
    % indices into a struct field.
    JettaMotion{i} = detectMotionSections(JettaData{i}.speed_fts);
    CamryMotion{i} = detectMotionSections(CamryData{i}.speed_fts);

    JettaData{i}.trial_ave_idle_rpm = ...
        mean(JettaData{i}.EngineRPM_rpm(JettaMotion{i}.idle_start_i:...
        JettaMotion{i}.idle_end_i));

    if show_ave_idle_rpms == true
        fprintf("Trial %d: \n",i);
        fprintf("Jetta trial ave rpm: %.2f\n",...
            JettaData{i}.trial_ave_idle_rpm);
    end

    speed = CamryData{i}.VehicleSpeed_MPH;
    rpm = CamryData{i}.EngineRPM_rpm;

    % select idle rpm where rpm is less than 900
    idle = rpm < 900;

    CamryData{i}.engine_running_idle_rpm = mean(rpm(idle));

    fprintf("Camry trial %d idle RPM: %.2f\n", ...
        i, CamryData{i}.engine_running_idle_rpm);
end
```

```
Trial 1:
Jetta trial ave rpm: 980.15
Camry trial 1 idle RPM: 682.47
Trial 2:
Jetta trial ave rpm: 880.22
Camry trial 2 idle RPM: 675.44
Trial 3:
```

Jetta trial ave rpm: 924.25
Camry trial 3 idle RPM: 698.35

Plot recorded trials (three trials on each graph)

```
% this folder is for saving all the plots automatically;
save_folder = ...
    "C:\Users\jiaxi\OneDrive\Documentos\GitHub\MATLAB-class-
project1\MOOMOO_CODE\New plots and reports";

% this is a vector containing all the file name for the plots, I hate
% typing all this, but it makes it easier later in the for loop
plot_filenames = [
    "CamryPos3Trial.png"
    "CamryVel3Trial.png"
    "CamryRawAccel3Trial.png"
    "CamryFilteredAccel3Trial.png"
    "CamryFilteredVel3Trial.png"
    "CamryRPMTime3Trial.png"
    "CamryIdle1RPMTime3Trial.png"
    "CamryAccelRPMTime3Trial.png"
    "CamryDecelRPMTime3Trial.png"
    "CamryIdle2RPMTime3Trial.png"
    "CamryRPMVel3Trial.png"
    "CamryIdle1RPMVel3Trial.png"
    "CamryAccelRPMVel3Trial.png"
    "CamryDecelRPMVel3Trial.png"
    "CamryIdle2RPMVel3Trial.png"
    "CamryFuelRate3Trial.png"
    "CamryFuelUsed3Trial.png"
    "JettaPos3Trial.png"
    "JettaVel3Trial.png"
    "JettaRawAccel3Trial.png"
    "JettaFilteredAccel3Trial.png"
    "JettaRPMTime3Trial.png"
    "JettaIdle1RPMTime3Trial.png"
    "JettaAccelRPMTime3Trial.png"
    "JettaDecelRPMTime3Trial.png"
    "JettaIdle2RPMTime3Trial.png"
    "JettaIdle1RPMVel3Trial.png"
    "JettaAccelRPMVel3Trial.png"
    "JettaDecelRPMVel3Trial.png"
    "JettaIdle2RPMVel3Trial.png"
    "JettaFuelRate3Trial.png"
    "JettaFuelUsed3Trial.png"
];

% this for loop will plot all the plots stated above, then save into the
% folder required;

% numel(plot_filenames) returns the size of the vector, it has 32 plots in
% total, so this just means 1:32
if plotting_enabled == 1
```

```
for graph = 1:numel(plot_filenames)
% for each of these statements, logic is like this:
% "graph" will scan from 1 to 32, and for each of the assigment below,
% it will first compare, if the graph is corresponding value, if it is,
% then the graph == corresponding value returns 1, which turns the
% plotting flag on, if not, it returns 0, which turns the plotting flag
% off, so when the "graph" scan thru 1 to 32, it will plot each plot
% once;
plot_camry_pos = (graph == 1);
plot_camry_vel = (graph == 2);
plot_camry_raw_accel = (graph == 3);
plot_camry_accel = (graph == 4);
plot_camry_filtered_vel = (graph == 5);
plot_camry_rpmtime = (graph == 6);
plot_camry_rpmtime_idle1 = (graph == 7);
plot_camry_rpmtime_accel = (graph == 8);
plot_camry_rpmtime_decel = (graph == 9);
plot_camry_rpmtime_idle2 = (graph == 10);
plot_camry_rpmvel = (graph == 11);
plot_camry_rpmvel_idle1 = (graph == 12);
plot_camry_rpmvel_accel = (graph == 13);
plot_camry_rpmvel_decel = (graph == 14);
plot_camry_rpmvel_idle2 = (graph == 15);
plot_camry_fuelratetime = (graph == 16);
plot_camry_fuelused = (graph == 17);
plot_jetta_pos = (graph == 18);
plot_jetta_vel = (graph == 19);
plot_jetta_accel = (graph == 20);
plot_jetta_filtered_accel = (graph == 21);
plot_jetta_rpmtime = (graph == 22);
plot_jetta_rpmtime_idle1 = (graph == 23);
plot_jetta_rpmtime_accel = (graph == 24);
plot_jetta_rpmtime_decel = (graph == 25);
plot_jetta_rpmtime_idle2 = (graph == 26);
plot_jetta_rpmvel_idle1 = (graph == 27);
plot_jetta_rpmvel_accel = (graph == 28);
plot_jetta_rpmvel_decel = (graph == 29);
plot_jetta_rpmvel_idle2 = (graph == 30);
plot_jetta_fuelratetime = (graph == 31);
plot_jetta_fuelused = (graph == 32);

% opens a window for plotting a plot;
fig = figure;

% this following for loop is just to loop thru trial 1 to trial 3 for
% each car's test data, and it will plot 3 trials on 1 plot for more
% compact plotting, and whether it will be plotted or not will be
% controlled by that if-statement in fron of each plotting code;

for i = 1:3
    % Camry: Distance vs time
    if plot_camry_pos
        plot(CamryData{i}.Time_sec,...
             CamryData{i}.trip_distanceft,'o',...
```

```
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Position vs Time');
    xlabel('Time (s)');
    ylabel('Distance (ft)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Speed vs time
if plot_camry_vel
    plot(CamryData{i}.Time_sec,...
         CamryData{i}.speed_fts,'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Speed vs Time');
    xlabel('Time (s)');
    ylabel('Speed (ft/s)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Raw acceleration from unfiltered speed
if plot_camry_raw_accel
    raw_accel = gradient(CamryData{i}.speed_fts,...
                         CamryData{i}.Time_sec);
    plot(CamryData{i}.Time_sec,...
         raw_accel,'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Raw Acceleration (from Speed) vs Time');
    xlabel('Time (s)');
    ylabel('Acceleration (ft/s^2)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Filtered acceleration vs time
if plot_camry_accel
    plot(CamryData{i}.Time_sec,...
         CamryData{i}.filtered_accel,'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
```

```
    title('Camry Filtered Acceleration vs Time');
    xlabel('Time (s)');
    ylabel('Acceleration (ft/s^2)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Kalman speed vs time
if plot_camry_filtered_vel
    plot(CamryData{i}.Time_sec,...
         CamryData{i}.filtered_speed,'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Kalman Speed vs Time');
    xlabel('Time (s)');
    ylabel('Speed (ft/s)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: RPM vs time
if plot_camry_rpmtime
    plot(CamryData{i}.Time_sec,...
         CamryData{i}.EngineRPM_rpm,'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Initial idle: RPM vs time
if plot_camry_rpmtime_idle1
    idx = CamryMotion{i}.idle_start_i:CamryMotion{i}.idle_end_i;
    plot(CamryData{i}.Time_sec(idx),...
         CamryData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Initial Idle: RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
```

```
    legend(TrialLegends,'Location','best');
end

% Camry: Acceleration: RPM vs time
if plot_camry_rpmtime_accel
    idx = CamryMotion{i}.accel_start_i:CamryMotion{i}.accel_end_i;
    plot(CamryData{i}.Time_sec(idx),...
         CamryData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Acceleration: RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Braking: RPM vs time
if plot_camry_rpmtime_decel
    idx = ...
        CamryMotion{i}.decel_start_i:numel(CamryData{i}.speed_fts);
    if isfield(CamryMotion{i},'decel_end_i')
        idx = ....
            CamryMotion{i}.decel_start_i:...
            CamryMotion{i}.decel_end_i;
    end
    plot(CamryData{i}.Time_sec(idx),...
         CamryData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Braking: RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Final idle: RPM vs time
if plot_camry_rpmtime_idle2 && ...
        isfield(CamryMotion{i}, 'second_idle_start_i')
    idx = CamryMotion{i}.second_idle_start_i:...
       CamryMotion{i}.second_idle_end_i;
    plot(CamryData{i}.Time_sec(idx),...
         CamryData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
```

```
    title('Camry Final Idle: RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: RPM vs speed
if plot_camry_rpmvel
    plot(CamryData{i}.speed_fts,...
         CamryData{i}.EngineRPM_rpm,'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Initial idle: RPM vs speed
if plot_camry_rpmvel_idle1
    idx = CamryMotion{i}.idle_start_i:CamryMotion{i}.idle_end_i;
    plot(CamryData{i}.speed_fts(idx),...
         CamryData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Initial Idle: RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Acceleration: RPM vs speed
if plot_camry_rpmvel_accel
    idx = CamryMotion{i}.accel_start_i:CamryMotion{i}.accel_end_i;
    plot(CamryData{i}.speed_fts(idx),...
         CamryData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Acceleration: RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
```

```
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Braking: RPM vs speed
if plot_camry_rpmvel_decel
    idx = ...
        CamryMotion{i}.decel_start_i:numel(CamryData{i}.speed_fts);
    if isfield(CamryMotion{i},'decel_end_i')
        idx = ....
            CamryMotion{i}.decel_start_i:...
            CamryMotion{i}.decel_end_i;
    end
    plot(CamryData{i}.speed_fts(idx),...
         CamryData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Braking: RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Final idle: RPM vs speed
if plot_camry_rpmvel_idle2 && ...
        isfield(CamryMotion{i}, 'second_idle_start_i')
    idx = CamryMotion{i}.second_idle_start_i:...
       CamryMotion{i}.second_idle_end_i;
    plot(CamryData{i}.speed_fts(idx),...
         CamryData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Final Idle: RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Fuel rate vs time
if plot_camry_fuelratetime
    plot(CamryData{i}.Time_sec,...
         CamryData{i}.fuelrate_lbs,'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
```

```
    title('Camry Fuel Rate vs Time');
    xlabel('Time (s)');
    ylabel('Fuel rate (lb/s)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Camry: Cumulative fuel vs time
if plot_camry_fuelused
    plot(CamryData{i}.Time_sec,...
         CamryData{i}.cumulated_fuel_lb,'o',...
        "MarkerFaceColor",CamryTrialColors(i), ...
        "MarkerEdgeColor",CamryTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Camry Cumulative Fuel vs Time');
    xlabel('Time (s)');
    ylabel('Fuel (lb)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Distance vs time
if plot_jetta_pos
    plot(JettaData{i}.Time_sec,...
         JettaData{i}.trip_distanceft - ...
         JettaData{i}.trip_distanceft(1),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Position vs Time');
    xlabel('Time (s)');
    ylabel('Distance (ft)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Speed vs time
if plot_jetta_vel
    plot(JettaData{i}.Time_sec,...
         JettaData{i}.speed_fts,'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Speed vs Time');
    xlabel('Time (s)');
    ylabel('Speed (ft/s)');
    set(gca,'FontSize',14);
    grid on;
```

```
    legend(TrialLegends,'Location','best');
end

% Jetta: Raw (logged) acceleration vs time
if plot_jetta_accel
    plot(JettaData{i}.Time_sec,...
         JettaData{i}.Acceleration_fts2,'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Raw Acceleration vs Time');
    xlabel('Time (s)');
    ylabel('Acceleration (ft/s^2)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Filtered acceleration vs time
if plot_jetta_filtered_accel
    plot(JettaData{i}.Time_sec,...
         JettaData{i}.filtered_accel,'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Filtered Acceleration vs Time');
    xlabel('Time (s)');
    ylabel('Acceleration (ft/s^2)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: RPM vs time
if plot_jetta_rpmtime
    plot(JettaData{i}.Time_sec,...
         JettaData{i}.EngineRPM_rpm,'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Initial idle: RPM vs time
if plot_jetta_rpmtime_idle1
    idx = JettaMotion{i}.idle_start_i:JettaMotion{i}.idle_end_i;
```

```
    plot(JettaData{i}.Time_sec(idx),...
         JettaData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Initial Idle: RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Acceleration: RPM vs time
if plot_jetta_rpmtime_accel
    idx = JettaMotion{i}.accel_start_i:JettaMotion{i}.accel_end_i;
    plot(JettaData{i}.Time_sec(idx),...
         JettaData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Acceleration: RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Braking: RPM vs time
if plot_jetta_rpmtime_decel
    idx = JettaMotion{i}.decel_start_i:...
        numel(JettaData{i}.speed_fts);
    if isfield(JettaMotion{i},'decel_end_i')
        idx = ...
            JettaMotion{i}.decel_start_i:...
            JettaMotion{i}.decel_end_i;
    end
    plot(JettaData{i}.Time_sec(idx),...
         JettaData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Braking: RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end
```

```
% Jetta: Final idle: RPM vs time
if plot_jetta_rpmtime_idle2 && ...
        isfield(JettaMotion{i}, 'second_idle_start_i')
    idx = JettaMotion{i}.second_idle_start_i:...
        JettaMotion{i}.second_idle_end_i;
    plot(JettaData{i}.Time_sec(idx),...
         JettaData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Final Idle: RPM vs Time');
    xlabel('Time (s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Initial idle: RPM vs speed
if plot_jetta_rpmvel_idle1
    idx = JettaMotion{i}.idle_start_i:JettaMotion{i}.idle_end_i;
    plot(JettaData{i}.speed_fts(idx),...
         JettaData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Initial Idle: RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Acceleration: RPM vs speed
if plot_jetta_rpmvel_accel
    idx = JettaMotion{i}.accel_start_i:JettaMotion{i}.accel_end_i;
    plot(JettaData{i}.speed_fts(idx),...
         JettaData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Acceleration: RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Braking: RPM vs speed
```

```
if plot_jetta_rpmvel_decel
    idx = JettaMotion{i}.decel_start_i:...
        numel(JettaData{i}.speed_fts);
    if isfield(JettaMotion{i},'decel_end_i')
        idx = ...
            JettaMotion{i}.decel_start_i:...
            JettaMotion{i}.decel_end_i;
    end
    plot(JettaData{i}.speed_fts(idx),...
         JettaData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Braking: RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Final idle: RPM vs speed
if plot_jetta_rpmvel_idle2 && ...
        isfield(JettaMotion{i}, 'second_idle_start_i')
    idx = JettaMotion{i}.second_idle_start_i:...
        JettaMotion{i}.second_idle_end_i;
    plot(JettaData{i}.speed_fts(idx),...
         JettaData{i}.EngineRPM_rpm(idx),'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Final Idle: RPM vs Speed');
    xlabel('Speed (ft/s)');
    ylabel('Engine speed (rpm)');
    set(gca,'FontSize',14);
    grid on;
    legend(TrialLegends,'Location','best');
end

% Jetta: Fuel rate vs time
if plot_jetta_fuelratetime
    plot(JettaData{i}.Time_sec,...
         JettaData{i}.instantFuelRategals,'o',...
        "MarkerFaceColor",JettaTrialColors(i), ...
        "MarkerEdgeColor",JettaTrialColors(i),...
        'MarkerSize',2.0);
    hold on;
    title('Jetta Fuel Rate vs Time');
    xlabel('Time (s)');
    ylabel('Fuel rate (gal/s)');
    set(gca,'FontSize',14);
    grid on;
```

```
            legend(TrialLegends,'Location','best');
        end

        % Jetta: Cumulative fuel vs time
        if plot_jetta_fuelused
            fuel_used = JettaData{i}.TripFuel_gal;
            first_valid = find(fuel_used>0,1);
            fuel_used = fuel_used - fuel_used(first_valid);
            fuel_used(1:first_valid-1) = NaN;
            plot(JettaData{i}.Time_sec,...
                 fuel_used,'o',...
                "MarkerFaceColor",JettaTrialColors(i), ...
                "MarkerEdgeColor",JettaTrialColors(i),...
                'MarkerSize',2.0);
            hold on;
            title('Jetta Cumulative Fuel vs Time');
            xlabel('Time (s)');
            ylabel('Fuel (gal)');
            set(gca,'FontSize',14);
            grid on;
            legend(TrialLegends,'Location','best');
        end
    end

    % this if statement will check if the plots actually contains lines,
    % for example, for some trials, the car does not stop at 0 MPH, so our
    % motion detection code cannot detect a second idle duration, so the
    % plot will be empty, this if statement account for that and skip the
    % exporting of that plot; "~" means "not"
    if ~isempty(findobj(fig,'Type','line'))
        % if plot not empty ,export the plots to our previous stated folder;
        exportgraphics(fig, fullfile(save_folder,...
            plot_filenames(graph)), ...
            'Resolution',300);
    else
        % if plot is empty, skip the plot and show the skipped file;
        fprintf('Skipped %s: no data for this section.\n',...
            plot_filenames(graph));
    end
    % close this window so that we can reopen another window for plotting
    % another plot, so we don't have to have 32 windows popping up when
    % running this code, it will just open, save, close one by one;
    close(fig);
    end
end
```

Save the workspace and its variables;

```
save("JettaData.mat");
save("CamryData.mat");
```

Published with MATLAB® R2024b